PicVerse

Social Media Mini Platform

Unstructured Database Lab Assignment
Project Idea 5

Tech Stack: MongoDB + Flask (Python) + HTML/CSS/JS

Members:

Indranil Roy(CSF 007)

Rajdeep Sarkar(CSF 008)

Saikat Pal(CSF 009)

Somnath Bera(CSF 010)

1. Project Overview

PicVerse is a basic Instagram-like social media platform built as part of the university database lab assignment. It demonstrates core database concepts including schema design, CRUD operations, aggregation pipelines, and indexing using MongoDB as the backend database.

Component	Technology
Database	MongoDB (Local — Community Edition)
Backend	Python 3 + Flask + PyMongo
Frontend	Plain HTML5 + CSS3 + Vanilla JavaScript
Auth	bcrypt password hashing, localStorage session
File Upload	Flask file handler, UUID-named local storage

2. Schema Design

The database uses 4 collections. The design decision between embedded and referenced documents was made based on query patterns and scalability.

2.1 Collections Overview

Collection	Primary Fields	Relationship	Approach
users	_id, username, email, bio, password	—	Root entity
posts	_id, user_id, caption, image_url, created_at	refs users	Referenced
comments	_id, post_id, user_id, text, created_at	refs posts + users	Referenced
likes	_id, post_id, user_id, created_at	refs posts + users	Referenced

2.2 Design Decisions: Embedded vs Referenced

- Comments are REFERENCED (not embedded in posts) — Comments can grow unboundedly per post. Embedding would cause documents to exceed MongoDB's 16MB limit and make pagination difficult.
- Likes are REFERENCED — Enables efficient aggregation queries like counting likes per post and finding top liked posts without loading entire post documents.
- Posts reference users via user_id — Users are a shared entity reused across posts. Embedding user data would cause denormalization and update anomalies.

2.3 Indexes

Indexes are created at application startup via db.py:

```
#users — enforce unique username and email
users.create_index([("username", ASCENDING)], unique=True)
```

```
users.create_index([("email", ASCENDING)], unique=True)

#posts — required by project spec, enables fast user feed queries
posts.create_index([("user_id", ASCENDING)])

#comments — fast lookup by post and by user
comments.create_index([("post_id", ASCENDING)])
comments.create_index([("user_id", ASCENDING)])

#likes — compound unique index prevents duplicate likes
likes.create_index([("post_id", ASCENDING), ("user_id", ASCENDING)], unique=True)
```

3. CRUD Operations

All CRUD routes are implemented as Flask Blueprints under the routes/ directory. Each collection has its own route file.

3.1 Users CRUD — routes/users.py

Method	Endpoint	Operation	Description
POST	/api/users/	Create	Register new user, bcrypt hash password
POST	/api/users/login	Read	Authenticate user, return id + username
GET	/api/users/	Read	Fetch all users (password stripped from response)
GET	/api/users/<id>	Read	Fetch single user by ObjectId
PUT	/api/users/<id>	Update	Update user fields, re-hash if password changed
DELETE	/api/users/<id>	Delete	Remove user document

3.2 Posts CRUD — routes/posts.py

Method	Endpoint	Operation	Description
POST	/api/posts/	Create	Create new post with user_id reference
GET	/api/posts/	Read	Fetch all posts sorted by created_at desc, joins username
GET	/api/posts/<id>	Read	Fetch single post by ObjectId
PUT	/api/posts/<id>	Update	Update post fields
DELETE	/api/posts/<id>	Delete	Remove post document

3.3 Comments CRUD — routes/comments.py

Method	Endpoint	Operation	Description
POST	/api/comments/	Create	Add comment referencing post_id and user_id
GET	/api/comments/post/<id>	Read	Get all comments for a post, joins username
DELETE	/api/comments/<id>	Delete	Remove a comment by ObjectId

3.4 Likes CRUD — routes/likes.py

Method	Endpoint	Operation	Description
POST	/api/likes/	Create	Like a post — unique index prevents duplicates
DELETE	/api/likes/	Delete	Unlike — removes by post_id + user_id pair
GET	/api/likes/count/<id>	Read	Count total likes for a specific post

4. Aggregation Pipelines

All aggregation logic is in routes/analytics.py. Three endpoints provide engagement analytics using MongoDB aggregation pipelines.

4.1 Most Liked Posts — /api/analytics/most-liked

Counts likes per post, sorts descending, takes top 5, and joins with the posts collection to retrieve captions.

```
likes.aggregate([
    {"$group": {"_id": "$post_id", "likeCount": {"$sum": 1}}},
    {"$sort": {"likeCount": -1}},
    {"$limit": 5},
    {"$lookup": {
        "from": "posts",
        "localField": "_id",
        "foreignField": "_id",
        "as": "post"
    }},
    {"$unwind": "$post"},
    {"$project": {"likeCount": 1, "caption": "$post.caption"}}
])
```

Operator	Purpose
$group + $sum	Aggregate like count per post_id
$sort	Order results by likeCount descending
$limit	Return only top 5 results
$lookup	Join likes with posts collection on _id
$unwind	Flatten the joined post array to single object

Operator	Purpose
$project	Select only likeCount and caption fields

4.2 Most Active Users — /api/analytics/active-users

Counts posts per user, sorts descending, takes top 5, and joins with users to retrieve usernames.

```
posts.aggregate([
    {"$group": {"_id": "$user_id", "postCount": {"$sum": 1}}},
    {"$sort": {"postCount": -1}},
    {"$limit": 5},
    {"$lookup": {
        "from": "users",
        "localField": "_id",
        "foreignField": "_id",
        "as": "user"
    }},
    {"$unwind": "$user"},
    {"$project": {"postCount": 1, "username": "$user.username"}}
])
```

4.3 Engagement per Post — /api/analytics/engagement

Two separate aggregations — one on likes, one on comments — are merged in Python by post_id to produce a combined engagement score.

```
# Likes per post
likes.aggregate([{"$group": {"_id": "$post_id", "count": {"$sum": 1}}}])

# Comments per post
comments.aggregate([{"$group": {"_id": "$post_id", "count": {"$sum": 1}}}])

# Merged in Python using dictionary union
all_ids = set(like_counts) | set(comment_counts)
result = [{"post_id": pid, "likes": like_counts.get(pid, 0),
           "comments": comment_counts.get(pid, 0)} for pid in all_ids]
```

5. Backend Architecture

5.1 Entry Point — app.py

Flask application factory. Registers all blueprints, serves uploaded files, and handles image upload via multipart form data.

```
app.register_blueprint(users_bp,     url_prefix="/api/users")
app.register_blueprint(posts_bp,     url_prefix="/api/posts")
app.register_blueprint(comments_bp,  url_prefix="/api/comments")
app.register_blueprint(likes_bp,     url_prefix="/api/likes")
app.register_blueprint(analytics_bp, url_prefix="/api/analytics")
```

```
@app.route('/api/upload', methods=['POST'])  # local image upload
@app.route('/uploads/<filename>')            # serve uploaded files
```

5.2 Database Connection — db.py

Single connection module imported by all route files. Creates indexes on first startup.

5.3 Authentication

- Passwords are hashed using bcrypt with auto-generated salt before storage.
- Login verifies password via bcrypt.checkpw() — plain text is never stored.
- On successful login, user id and username are stored in browser localStorage as a session object.
- All post/like/comment operations read the session from localStorage — no manual ID entry required.

6. Frontend

6.1 Structure

File	Purpose
index.html	Single page app — login screen, sidebar nav, all sections, post modal
style.css	Dark theme UI with CSS variables, responsive grid, animations
app.js	All JS logic — auth, API calls, DOM manipulation, session management

6.2 Pages / Sections

Section	Features
Login / Register	Toggle between sign in and register. Auto-login after registration.
Feed	Post cards grid. Click to open post modal.
Post Modal	Full post view with like/unlike toggle and live comment section.
New Post	Create post with caption. URL or local file upload toggle for image.
Users	Create new users. View all registered users.
Analytics	Most liked posts, most active users, engagement per post.

7. Steps to Run the App

Step 1 — Start MongoDB

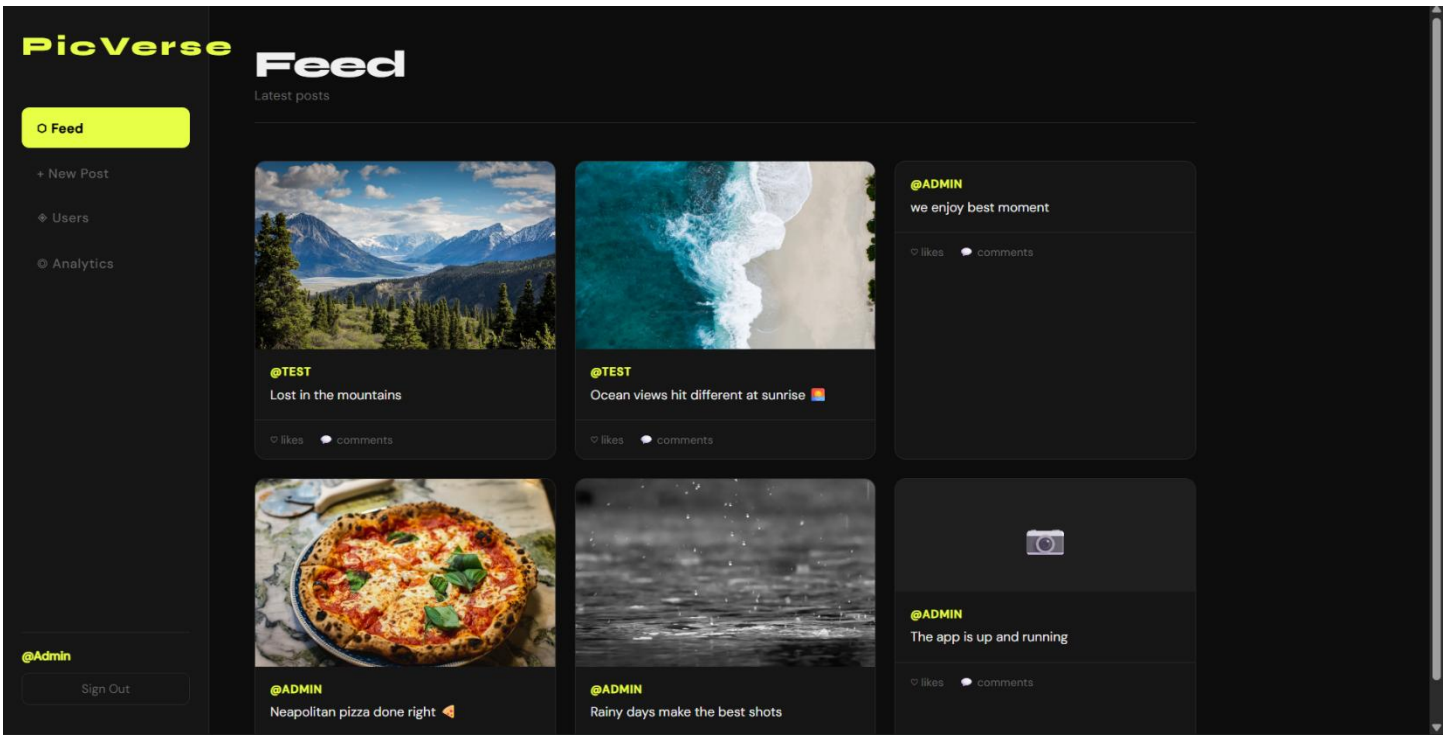
```
net start MongoDB   #run as Administrator
```

Step 2 — Start Flask Backend

```
cd backend
pip install flask pymongo flask-cors bcrypt
python app.py
```

```
#Server runs on http://127.0.0.1:5000
```

Step 3 — Start Frontend Server

```
cd frontend
python -m http.server 8080
#Open http://localhost:8080 in browser
```

LIVE APP VIEW